Epoch-Stamped Trailing (#5): What It Is, How Rice Implements It, and Why It Doesn't Fire

Rice project notes

Abstract

This note documents `OPT_EPOCH_TRAIL`, optimization #5 from `backtracking-optimizations.md`: a WAM-style trail check adapted to Rice's graph-reduction runtime. It is implemented, correct, and measured (`tests/bench.sh`) to have essentially zero effect on every benchmark tried, including a microbenchmark deliberately shaped to trigger it. This is not a bug and not (only) a fact about the specific programs tried — it follows from a structural property of graph reduction with sharing that the WAM does not have. We keep the optimization in the codebase, off by default, as a baseline for eventually measuring how much each backtracking optimization actually contributes.

1 The optimization in Prolog

1.1 The trail

In the WAM (Warren Abstract Machine), a logic variable is a heap cell that starts unbound and gets *bound* by unification. Backtracking to an earlier choice point must undo every binding made since that choice point was created, so that the alternative clause sees the variables in their original, unbound state. The *trail* is the stack of addresses that were bound since the last choice point: backtracking pops the trail back down to the choice point's own mark, resetting each recorded address back to unbound.

Not every binding needs to go on the trail, though. Consider:

```
member(X, [X|_]).
member(X, [_|T]) :- member(X, T).

?- member(X, [1,2,3]).
X = 1 ;
X = 2 ;
X = 3.
```

Each retry of `member` creates a fresh activation: a fresh copy of `X` (renamed apart) and a fresh cons cell for the recursive call. Binding *that* fresh `X` to `2` on the second solution does not need trailing in the way binding some *pre-existing* variable would — but the WAM's actual criterion is not "was this variable created during this clause's activation", it's something more precisely mechanical and cheaper to check.

1.2 The heap-address / choice-point-age check

The WAM's heap is a bump-allocated stack: every new term, cell, or unbound variable is allocated at the current heap-top pointer, which only ever increases during forward execution. Every choice

point records `HB` — the heap-top pointer *at the moment the choice point was created*. Backtracking to a choice point doesn't just pop trail entries; it also resets the heap pointer itself back down to what it was when the choice point was created, in one $O(1)$ step, discarding (making available for reuse) every cell allocated after that point.

This gives a cheap, purely address-based test for whether a binding needs trailing at all: when binding a variable at heap address a, while the active choice point's mark is HB,

- if $a < HB$ (the variable existed *before* the current choice point), the binding must be trailed — the variable will still exist, unbound, after a heap reset past this choice point, and something outside this choice's exploration may still read it;
- if $a \geq HB$ (the variable was created *after* the current choice point), the binding needs no trail entry at all — backtracking is going to reset the heap pointer back past a regardless, which makes the cell (bound or not) vanish along with everything else allocated in this exploration. There is no scenario in which anything outside this choice's own exploration could still hold a valid reference to it afterward.

This is a real, load-bearing optimization in WAM implementations: a very large fraction of all unification in a typical Prolog program is binding variables that were *just* created as part of the current clause activation, and skipping the trail push for all of them is a large, constant-factor win, because in Prolog *every* data construction and argument-passing step goes through this same bind-and-maybe-trail machinery. There is no "fast path" for purely deterministic computation in the WAM the way there is in Rice (Section 3) — every unification is a potential trail candidate, deterministic or not, so an age-based filter is cutting a very large population down.

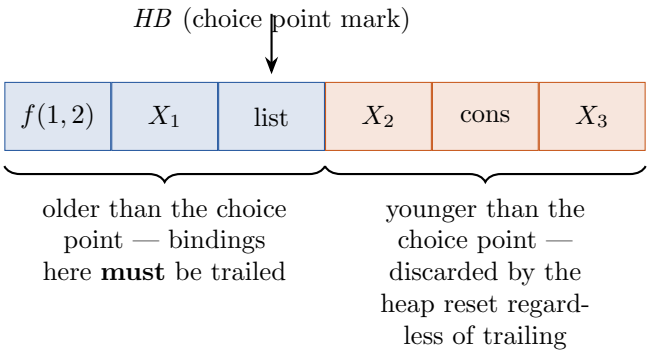


Figure 1: The WAM heap at the moment of a choice point (mark HB) and after some further, forward computation. Bindings to the blue (pre-existing) cells need trail entries; bindings to the orange (freshly allocated since the choice point) cells don't, because backtracking's own heap-pointer reset discards them unconditionally.

This check is sound in the WAM specifically because of the *binding convention*: unification always binds the *younger* variable to the *older* term, never the reverse. If an old, surviving variable were ever bound to point at a young, about-to-be-discarded cell, resetting the heap past that young cell would leave the old variable dangling. The WAM's binding-order rule exists specifically to make the age-based trail shortcut sound.

2 How Rice implements the analogous check

Rice has no bump-allocated heap and no single global "pointer reset" operation — nodes are individually heap-allocated (`alloc()`, `runtime/memory.h`) and never explicitly freed on backtrack. So there is no address-comparison-against-a-heap-mark to borrow directly. Instead, `backtracking-optimizations.md`'s #5 adapts the same idea using an explicit logical clock:

- A global counter, `current_epoch` (`runtime.c`), starts at 0 and is incremented once per choice point created — every call to `choose()` (resolving a live `?`) or `push_choice()` (resolving a free variable against one of several constructors).
- Every node, at the moment it is *freshly allocated*, is stamped with the counter's current value: a new `epoch` field on `Node` (`node.h`, compiled in only under the flag). This needed finding *every* node-allocating site, not just the obvious ones: `Rice.Codegen.C.ToH`'s generated `make_X`/`make_X_free` constructor/free-variable templates, and `runtime.h`/`runtime.c`'s own hand-written `make_forward`, `make_choice`, `free_var`, `make_apply{1,2,3}`, `copy()`, and `set_copy_free()`.
- `push_node` — the function that decides whether a node about to be mutated needs a trail entry at all (already gated by #1 on `nondet`) — adds the age check on top:

```
static inline void push_node(union field root, bool nondet)
{
    root.n->nondet = nondet;
#if OPT_EPOCH_TRAIL
    if(nondet && root.n->epoch < current_epoch)
#else
    if(nondet)
#endif
    {
        stack_push(bt_stack, root, copy(root), false);
    }
}
```

`root.n->epoch < current_epoch` is Rice's version of "$a < HB$": the node was allocated before the currently-active choice point, so it might be read again after backtracking and must be trailed. When the node's epoch is not older than the active choice (`epoch` $\geq$ `current_epoch`), it was allocated *as part of* exploring the current alternative, and #5's claim is the same as the WAM's: nothing outside this exploration could hold a valid reference to it, so skip the trail push.

The stack-frame bookkeeping needed to make this correct across backtracking is unconditional, not compiled out when the flag is off, so that `choose()`/`push_choice()` are the only two places that need to know the flag exists for the stack half of this at all: `FieldPair` (`stack.h`) carries an `epoch` field, snapshotted on every `stack_push` and restored on every `pop`/`pop_to` (`stack.c`), regardless of `OPT_EPOCH_TRAIL`.

3 Why it doesn't fire

Measured directly (`tests/bench.sh`, `STATUS.md`'s 2026-09-12 entry): across all ten `SLOW`-tagged benchmarks, `OPT_EPOCH_TRAIL` produces *bit-for-bit identical* total allocation counts to the shipped

defaults, and is consistently 3–15% *slower* (pure bookkeeping overhead, no payoff). A microbenchmark built specifically to hit #5's precondition — one long-lived outer choice, with 5000 recursive steps underneath it each reading a second, already-resolved choice — *also* shows zero difference. This isn't several unlucky test programs; there's a structural reason.

3.1 `undo()` restores a whole node, pointers included

```
bool undo(int cutoff)
{
    ...
    do
    {
        frame = pop(bt_stack);
        if(frame->lhs.n != NULL)
        {
            memcpy(frame->lhs.n, frame->rhs.n, sizeof(Node));
        }
    } while(!frame->choice && size(bt_stack) > cutoff);
    ...
}
```

Restoring a trailed node's saved snapshot overwrites *all* of its fields, including whatever child pointers it held. So restoring *one* mutated ancestor automatically discards access to everything that was built *underneath* it during the abandoned exploration — regardless of whether any of those descendant nodes were individually trailed at all. The only node that ever needs its own trail entry, for any given piece of nested computation, is the *outermost* node that is both mutated and directly touches nondeterminism.

3.2 That outermost node is always older than what's nested inside it

This is the point that closes off #5's opportunity almost entirely: to even *reach* a choice nested three function calls deep, something has to already exist and already be under evaluation to make those three calls. That something's own reduction is what the case-dispatch or apply-resolution machinery marks `nondet = true` on, and it was, by definition, allocated *before* the nested choice it eventually reaches. It can never be "young" relative to a choice nested inside its own evaluation — the containment relationship forces the age ordering the other way around. For #5 to find anything to skip, a node would need to be simultaneously (a) directly, independently nondet-tainted by touching some choice/free variable in its *own* right, and (b) younger than a *different*, already-active outer choice — i.e. genuinely separate, parallel nondeterminism nested inside an active choice's scope, not merely "deep recursion under a choice", no matter how deep.

3.3 Graphs

Figure 2 shows the shape of the microbenchmark's graph while evaluating one branch of the outer choice: an old, top-level expression node `M` (older than the active choice, because it had to already exist and be forced to reach it) dominates a long chain of freshly-built application nodes from unrolling `loop`. Every node in that chain is young relative to the active choice, but *none of them is individually nondet* — reading the resolved value of `f` through an ordinary argument doesn't propagate a taint onto the node built from it; only `f`'s own node (marked directly by `choose()`)

and `M` (marked because *its* own reduction dispatches directly on a live choice) are ever candidates for the trail at all.

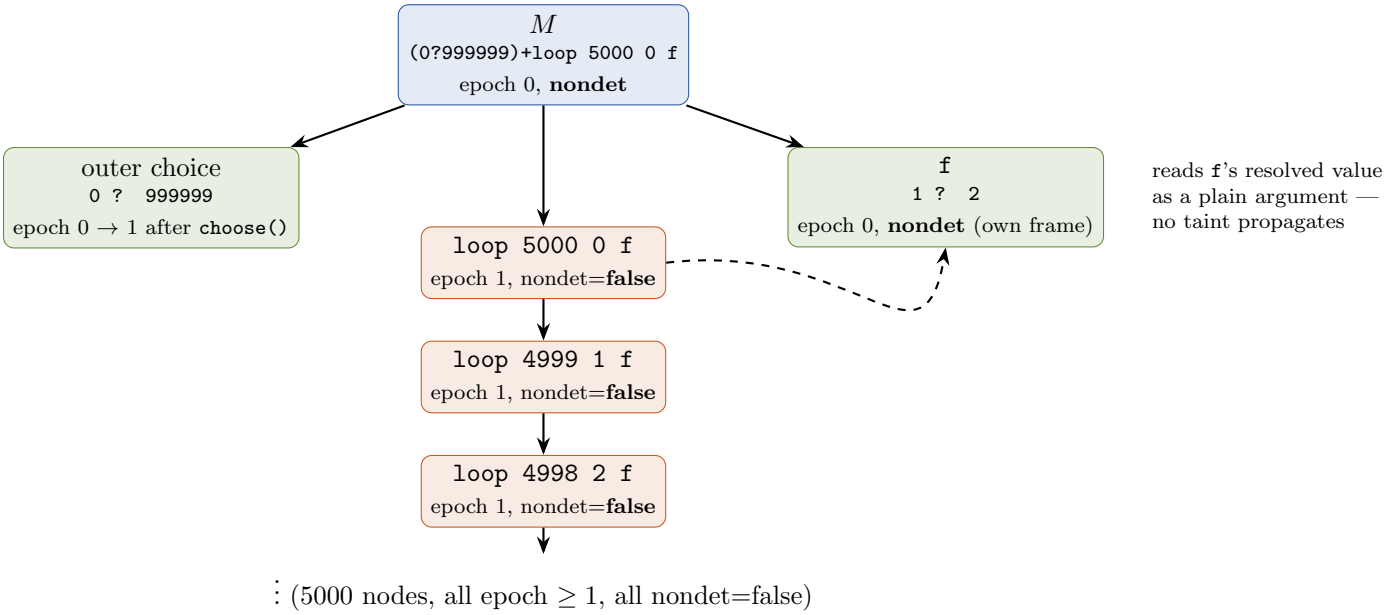


Figure 2: The microbenchmark's graph while exploring one branch. `M` (blue) is the one node whose own reduction directly dispatches on a live choice, so it is the one node #1 marks `nondet` and pushes to the trail. The entire `loop` chain (orange) is young relative to the active choice, but every node in it has `nondet=false` — #1 already excludes all of them from the trail, so #5's age check never even gets a candidate to filter.

Figure 3 shows what `undo()` does when backtracking reaches the outer choice: popping `M`'s trail frame overwrites `M`'s fields with its saved, pre-evaluation snapshot — including the pointer that used to lead into the `loop` chain. That whole chain becomes unreachable in one step, independent of whether any node in it was ever individually trailed.

3.4 The contrast with Prolog, restated

In the WAM, *every* unification — deterministic or not — goes through the same bind-and-maybe-trail path, so an age filter is cutting a large, otherwise-undifferentiated population down to size. Rice's #1 already removes purely deterministic reduction from the trail machinery entirely (Figure 2's whole `loop` chain is an example: 5000 nodes, zero trail candidates, with or without #5). What's left over after #1 is a small population that is, by the containment argument above, already dominated by nodes old enough to need trailing regardless of #5's check. Rice gets most of what the WAM's optimization buys Prolog for free, via sharing and #1, rather than needing the explicit age check to buy a second, separate slice of it.

4 Status

Kept in the codebase, `OPT_EPOCH_TRAIL` defaulting off, rather than removed: the goal per `backtracking-optimizations.md` is eventually reporting how much *each* backtracking optimization contributes empirically (`tests/bench.sh`), and a documented, measured "this one doesn't help, here is why" is

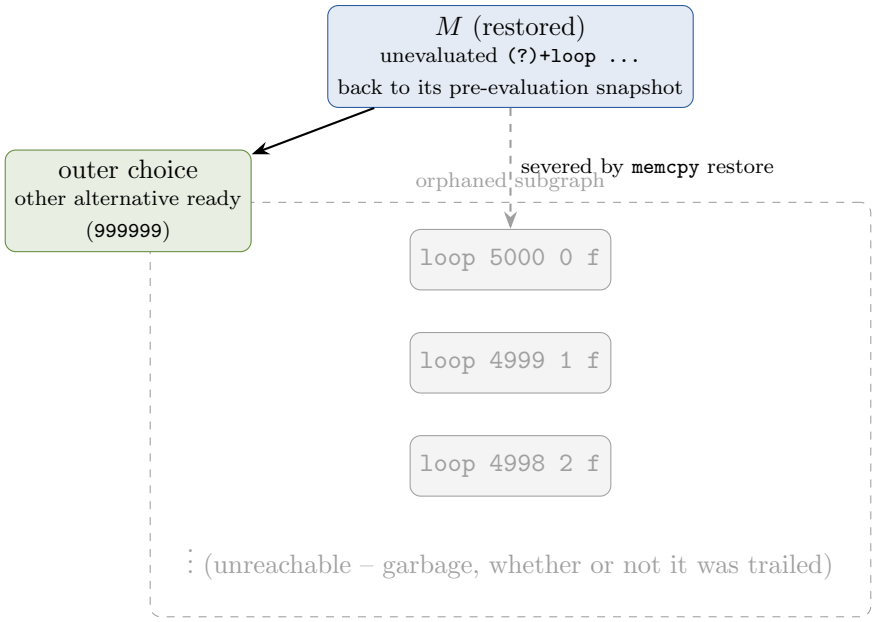


Figure 3: After `undo()` pops `M`'s frame and backtracks to the outer choice: `M` is restored to its exact pre-evaluation content, including its pointer fields, which no longer lead into the `loop` chain at all. Whatever was built during this exploration is orphaned in one step. Nothing needed to check `loop`'s nodes' individual ages for this to be correct – they were never going to be read again either way.

itself a useful data point for that report, not dead weight to delete. If a workload is ever found with genuinely separate, nested nondeterminism inside an already-active choice's scope — something other than the generate-then-deterministically-verify shape common to `old_rice/benchmarks/` — it would be a natural candidate to re-test #5 against.